Legged Robots Practical

03.10.2023

Plan

Single Leg

- **W1:** Introduction
- **W2:** Derivation of double pendulum's kinematics and dynamics
- **W3:** Jacobian (Cartesian PD + Force Control)
- **W4:** Inverse Kinematics (compare with force control)
- **W5:** Single-leg hopping

Biped

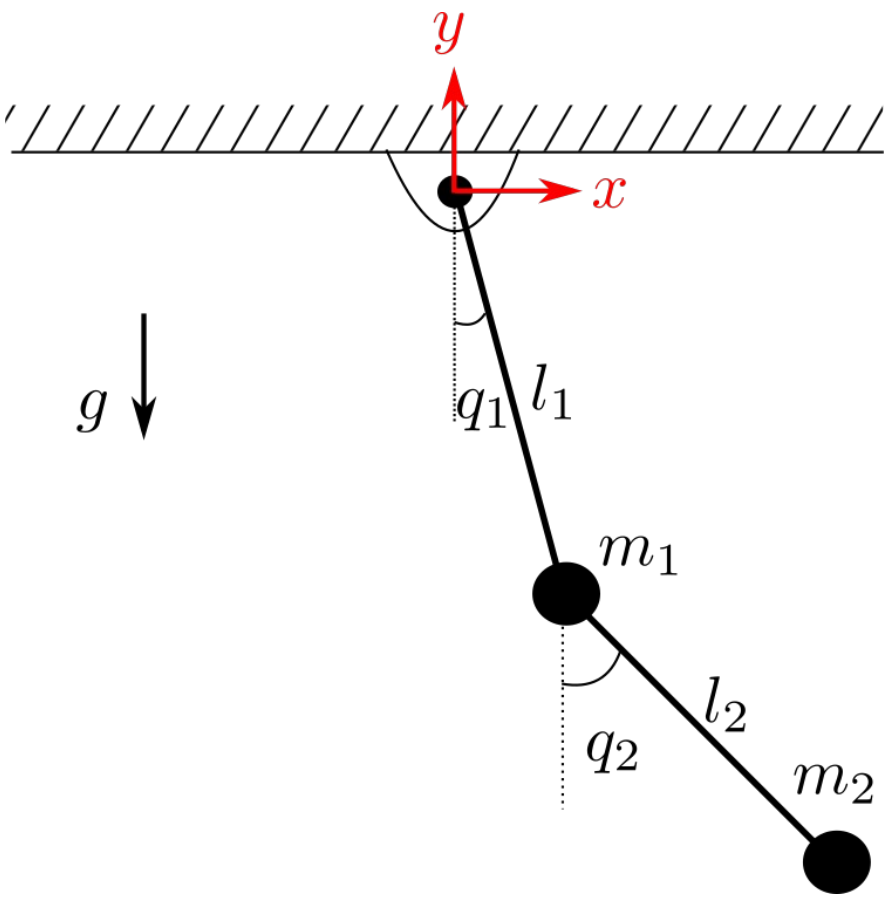
- **W6-8:** Biped Walking with Divergent Component of Motion (BD Atlas) **[Mini-Report]**

Quadruped

- **W9-10:** Quadruped CPG Trot
- **W11-14:** Quadruped Locomotion Project (CPGs, Deep RL) **[Report - Quadruped]**

Jacobian

Recall: Double Pendulum Forward Kinematics



$$x_1(q_1) = l_1 sin(q_1(t))$$

$$y_1(q_1) = -l_1 cos(q_1(t))$$

$$x_2(q_1, q_2) = l_1 sin(q_1(t)) + l_2 sin(q_2(t))$$

$$y_2(q_1, q_2) = -l_1 cos(q_1(t)) - l_2 cos(q_2(t))$$

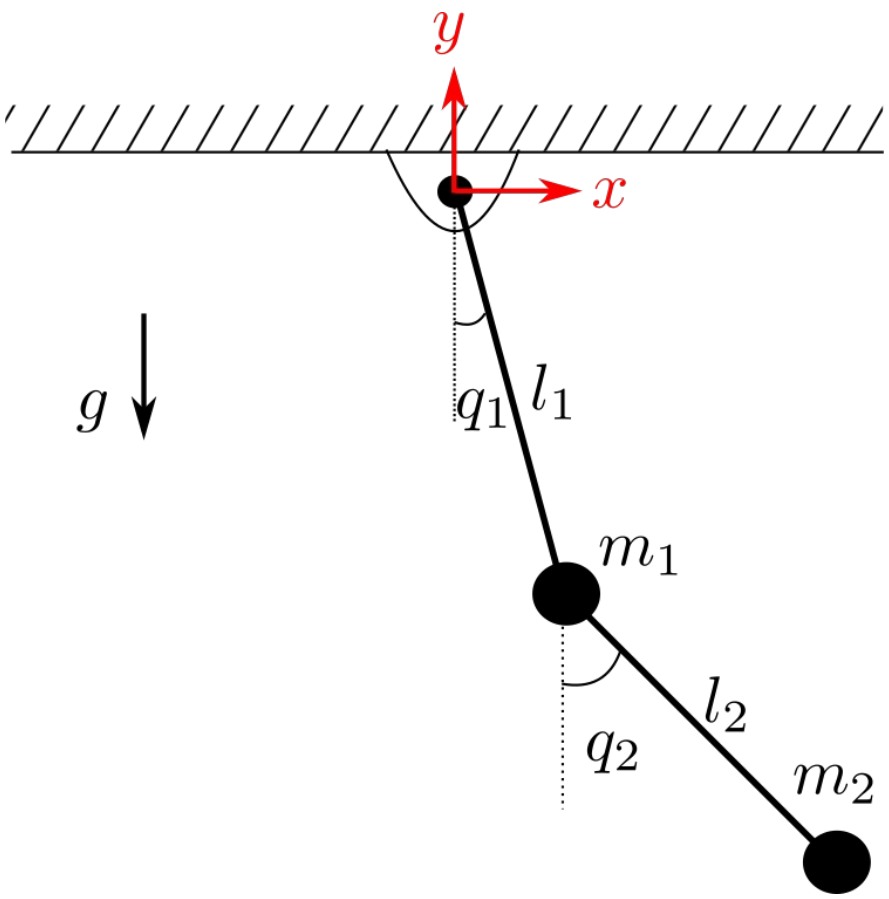
$$\dot{x}_1(q_1, \dot{q}_1) = l_1 cos(q_1(t))\dot{q}_1(t) \qquad \dot{x} = \frac{dx}{dt}$$

$$\dot{y}_1(q_1, \dot{q}_1) = l_1 sin(q_1(t))\dot{q}_1(t)$$

$$\dot{x}_2(\dots) = l_1 cos(q_1(t))\dot{q}_1(t) + l_2 cos(q_2(t))\dot{q}_2(t)$$

$$\dot{y}_2(\dots) = l_1 sin(q_1(t))\dot{q}_1(t) + l_2 sin(q_2(t))\dot{q}_2(t)$$

Rewrite



$$x_1(q_1) = l_1 sin(q_1(t))$$

$$y_1(q_1) = -l_1 cos(q_1(t))$$

$$x_2(q_1, q_2) = l_1 sin(q_1(t)) + l_2 sin(q_2(t))$$

$$y_2(q_1, q_2) = -l_1 cos(q_1(t)) - l_2 cos(q_2(t))$$

$$\dot{x}_1(q_1, \dot{q}_1) = l_1 cos(q_1(t))\dot{q}_1(t) \qquad \dot{x} = \frac{dx}{dt}$$

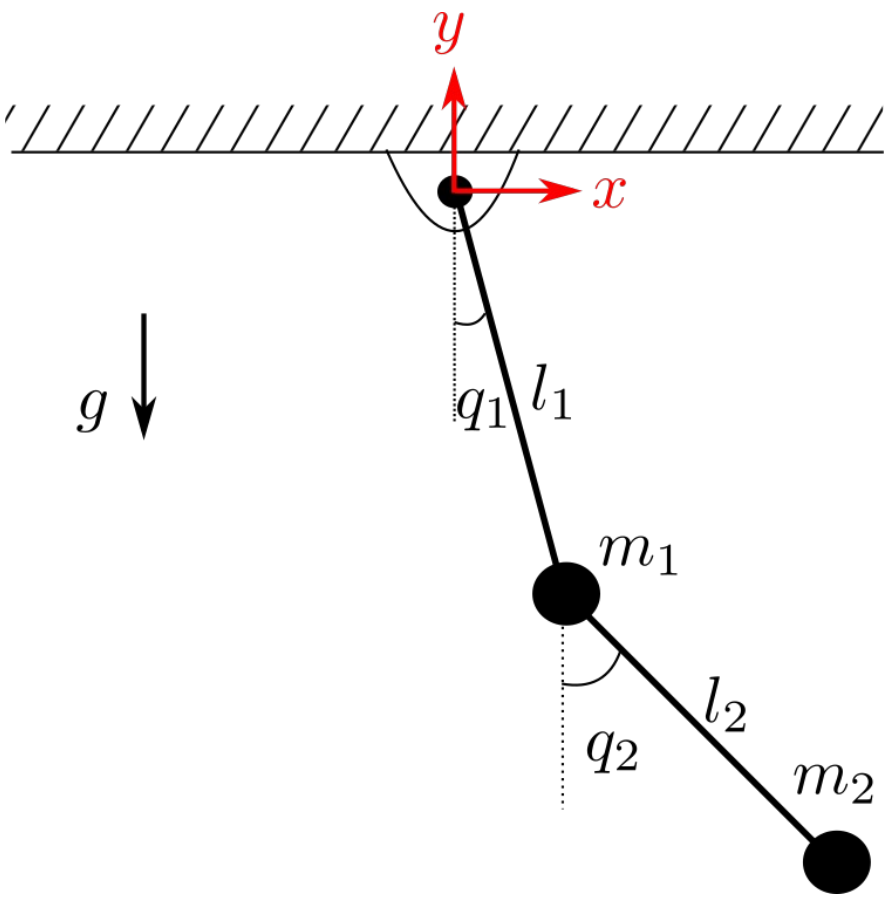
$$\dot{y}_1(q_1, \dot{q}_1) = l_1 sin(q_1(t))\dot{q}_1(t)$$

$$\dot{x}_2(\dots) = l_1 cos(q_1(t))\dot{q}_1(t) + l_2 cos(q_2(t))\dot{q}_2(t)$$

$$\dot{y}_2(\dots) = l_1 sin(q_1(t))\dot{q}_1(t) + l_2 sin(q_2(t))\dot{q}_2(t)$$

$$\begin{bmatrix} \dot{x}_2 \\ \dot{y}_2 \end{bmatrix} = \begin{bmatrix} l_1 cos(q_1) & l_2 cos(q_2) \\ l_1 sin(q_1) & l_2 sin(q_2) \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}$$

Jacobian



$$\begin{bmatrix} \dot{x}_2 \\ \dot{y}_2 \end{bmatrix} = \begin{bmatrix} l_1 cos(q_1) & l_2 cos(q_2) \\ l_1 sin(q_1) & l_2 sin(q_2) \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}$$

$$v = J(q)\dot{q}$$

$$J(q) = \begin{bmatrix} l_1 cos(q_1) & l_2 cos(q_2) \\ l_1 sin(q_1) & l_2 sin(q_2) \end{bmatrix}$$

Jacobian

- Matrix equivalent of the derivative
  - The derivative of a vector-valued function with respect to a vector

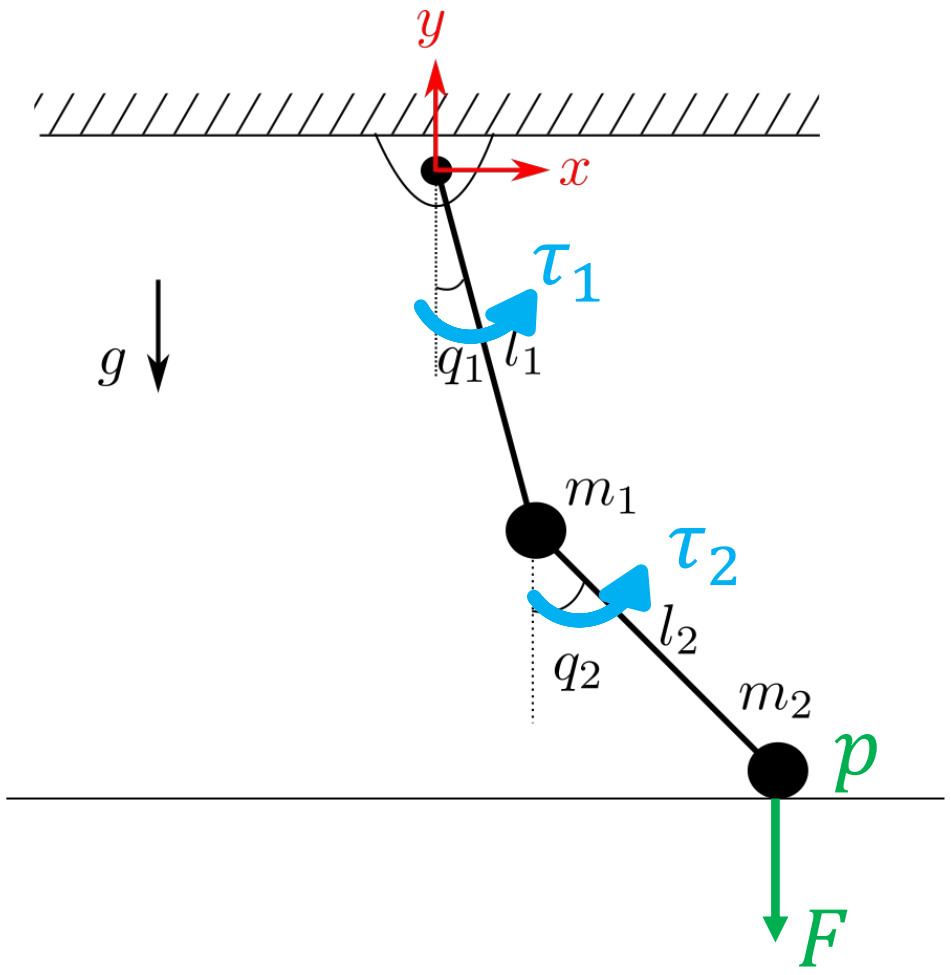
$$\boldsymbol{y} = F(\boldsymbol{x}) \qquad \boldsymbol{x} \in \mathbb{R}^n, \boldsymbol{y} \in \mathbb{R}^m$$

- The Jacobian is an *m* x *n* matrix

$$\boldsymbol{J} = \frac{\partial \boldsymbol{F}}{\partial \boldsymbol{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$

How can we use the Jacobian to control our system?

$$\boxed{\boldsymbol{v} = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}}}$$



- If we want to apply a force $\boldsymbol{F}$ at the end effector, we can compute the corresponding desired joint torques $\boldsymbol{\tau}$ with:

$$\boldsymbol{\tau} = \boldsymbol{J}^T(\boldsymbol{q})\boldsymbol{F}$$

- Why? Principle of Virtual Work
  - For a virtual displacement $\delta\boldsymbol{q}$ at the joints, the virtual displacement $\delta\boldsymbol{p}$ at the end effector is:

$$\delta\boldsymbol{p} = \boldsymbol{J}(\boldsymbol{q})\delta\boldsymbol{q}$$

  - Work done by $\boldsymbol{F}$:

$$\boldsymbol{F}^T\delta\boldsymbol{p} = \boldsymbol{F}^T\boldsymbol{J}(\boldsymbol{q})\delta\boldsymbol{q}$$

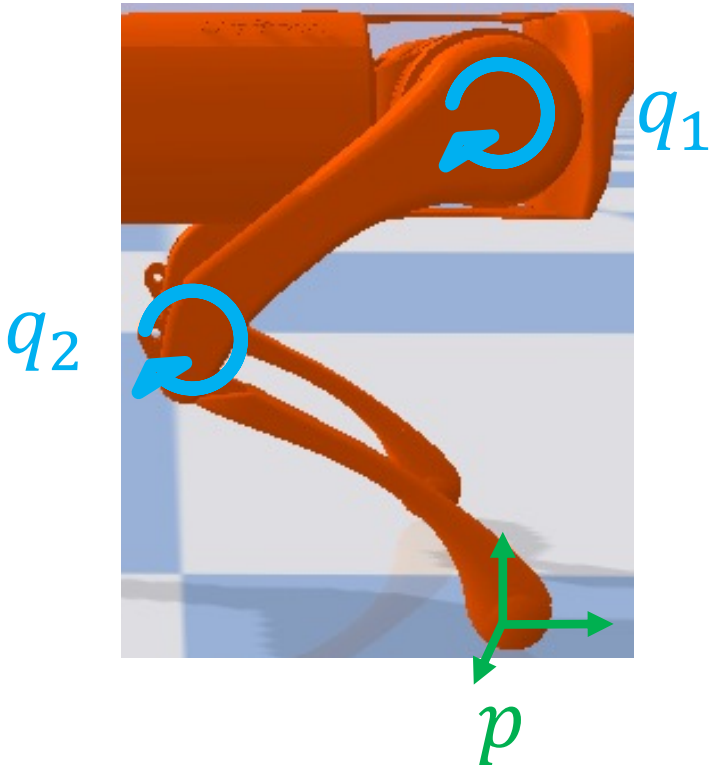
  - Work done by joint torques $\boldsymbol{\tau}$:

$$\boldsymbol{\tau}^T\delta\boldsymbol{q}$$

  - Work done must be equal

$$\boldsymbol{\tau}^T\delta\boldsymbol{q} = \boldsymbol{F}^T\boldsymbol{J}(\boldsymbol{q})\delta\boldsymbol{q} = \left(\boldsymbol{J}^T(\boldsymbol{q})\boldsymbol{F}\right)^T\delta\boldsymbol{q}$$

Summary: Joint angles ←→ Cartesian space



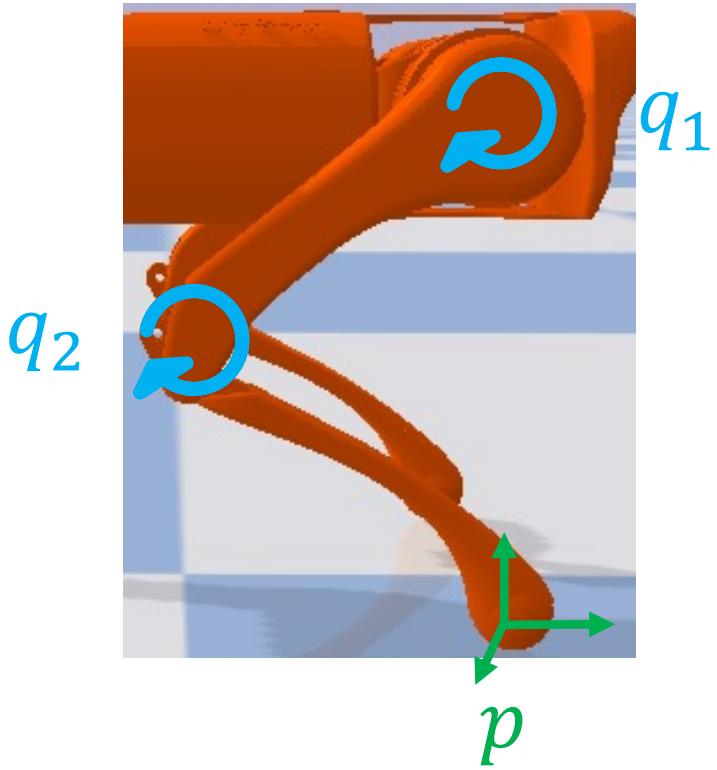
$p = f(q)$ Forward kinematics

$q = f^{-1}(p)$ Inverse kinematics

$\dot{p} = v = J(q)\dot{q}$ Foot linear velocity

$\tau = J^T(q)F$ Map desired end effector force to torques

Leg Control



$$p = f(q)$$ Forward kinematics

$$q = f^{-1}(p)$$ Inverse kinematics

$$\dot{p} = v = J(q)\dot{q}$$ Foot linear velocity

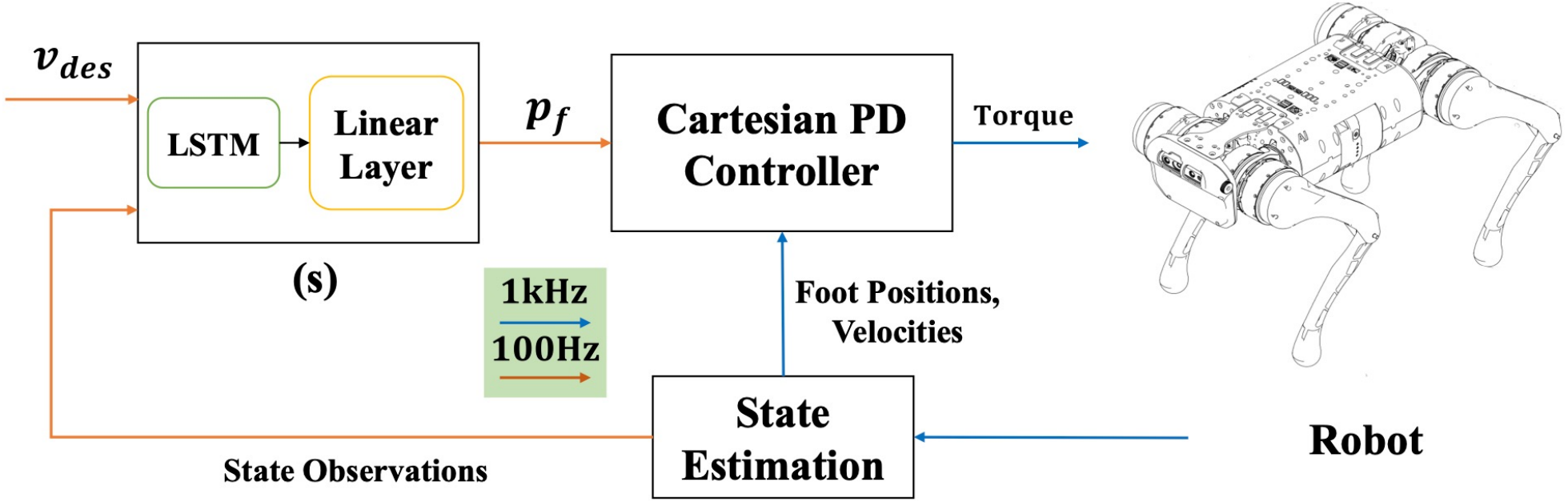
$$\tau = J^T(q)F$$ Map desired end effector force to torques

$$\tau_{joint} = K_{p,joint}(q_d - q) + K_{d,joint}(\dot{q}_d - \dot{q})$$ Joint PD

$$\tau_{Cartesian} = J^T(q)\left[K_{p,Cartesian}(p_d - p) + K_{d,Cartesian}(v_d - v)\right]$$ Cartesian PD

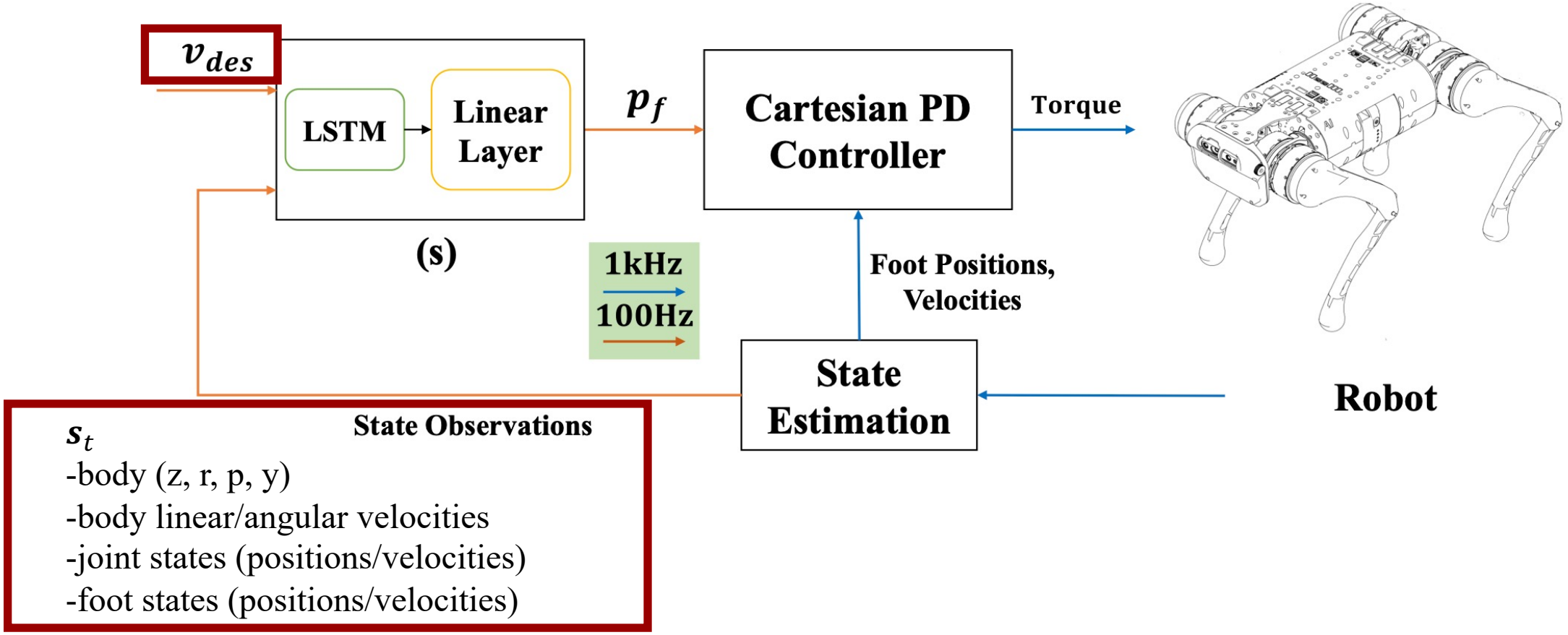
$$\tau_{final} = \tau_{joint} + \tau_{Cartesian} + J^T(q)F$$ Total applied torque

Application: Quadruped Running with Reinforcement Learning



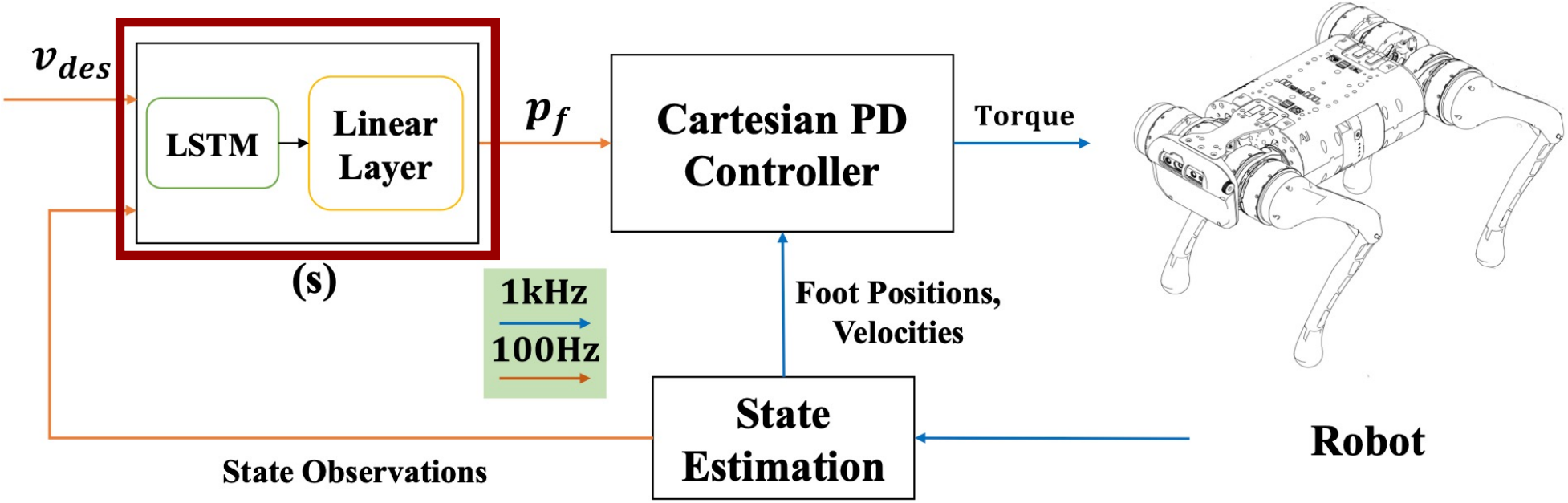
G. Bellegarda, Y. Chen, Z. Liu, Q. Nguyen. *Robust High-Speed Running for Quadruped Robots via Deep Reinforcement Learning*. IROS 2022

Application: Quadruped Running with Reinforcement Learning



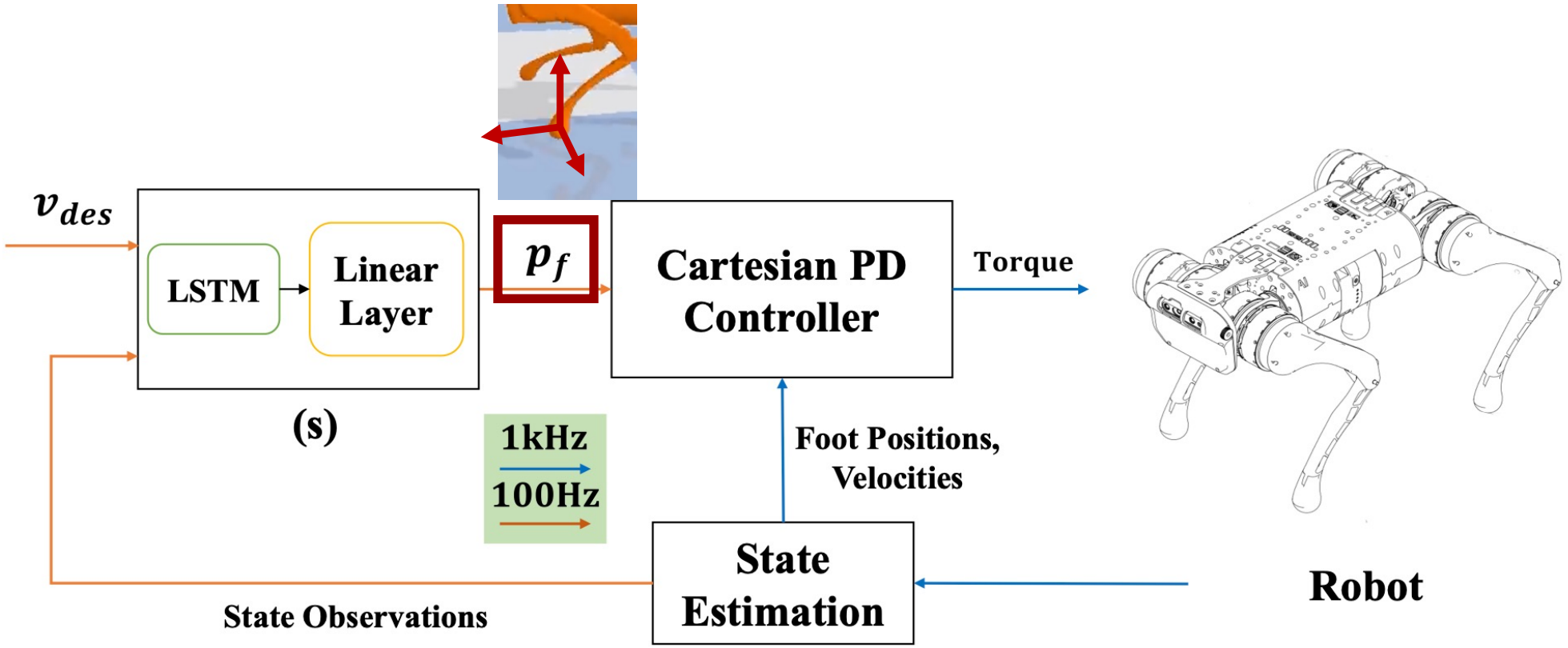
G. Bellegarda, Y. Chen, Z. Liu, Q. Nguyen. *Robust High-Speed Running for Quadruped Robots via Deep Reinforcement Learning*. IROS 2022

Application: Quadruped Running with Reinforcement Learning



G. Bellegarda, Y. Chen, Z. Liu, Q. Nguyen. *Robust High-Speed Running for Quadruped Robots via Deep Reinforcement Learning*. IROS 2022

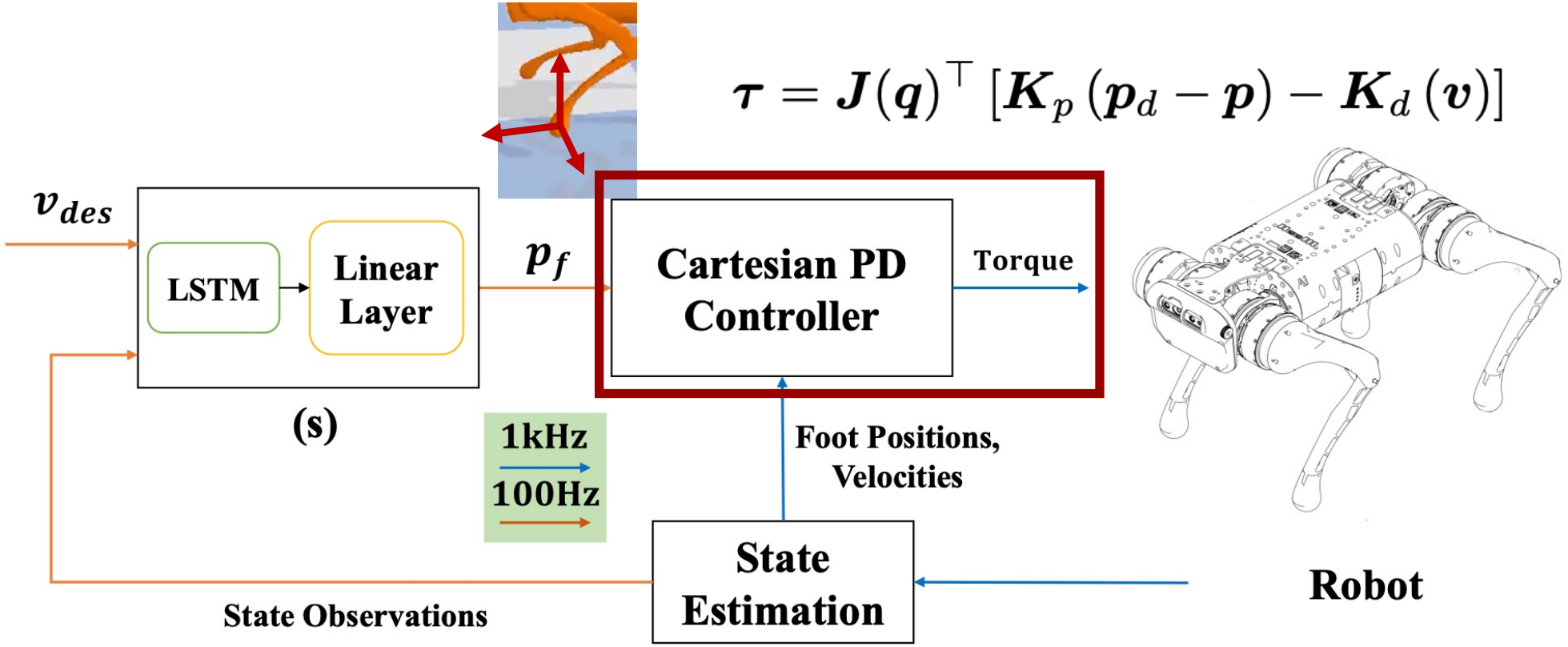
Application: Quadruped Running with Reinforcement Learning



G. Bellegarda, Y. Chen, Z. Liu, Q. Nguyen. *Robust High-Speed Running for Quadruped Robots via Deep Reinforcement Learning*. IROS 2022

Application: Quadruped Running with Reinforcement Learning

$$\boldsymbol{\tau} = \boldsymbol{J}(\boldsymbol{q})^\top \left[\boldsymbol{K}_p\left(\boldsymbol{p}_d - \boldsymbol{p}\right) - \boldsymbol{K}_d\left(\boldsymbol{v}\right)\right]$$

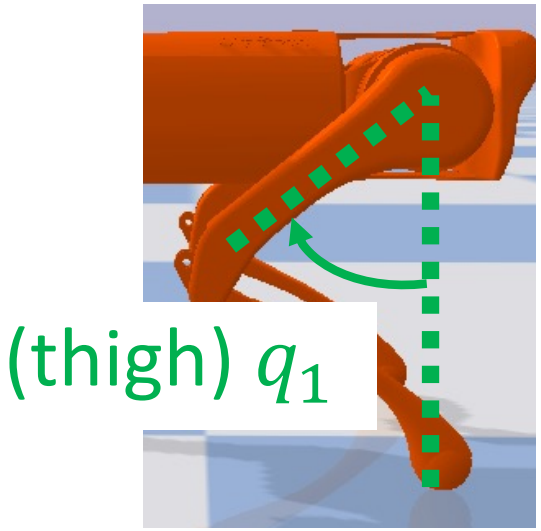


G. Bellegarda, Y. Chen, Z. Liu, Q. Nguyen. *Robust High-Speed Running for Quadruped Robots via Deep Reinforcement Learning*. IROS 2022

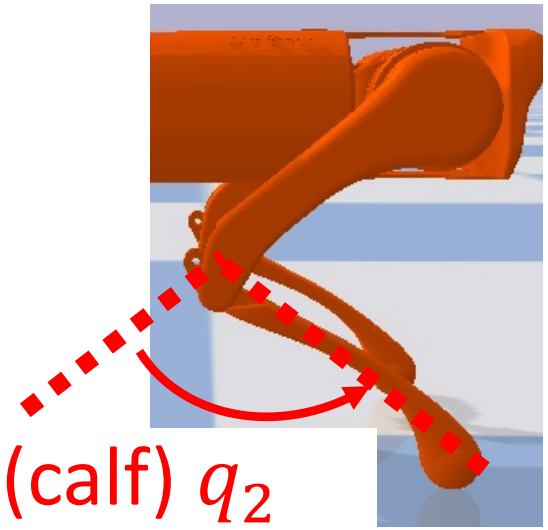
Application: Quadruped Running with Reinforcement Learning

G. Bellegarda, Y. Chen, Z. Liu, Q. Nguyen. *Robust High-Speed Running for Quadruped Robots via Deep Reinforcement Learning*. IROS 2022

Pybullet Quadruped Leg Joint References



- Careful: what is the axis?
- In figure, q_1 is $\pi/4$ (axis into page)
- In figure, q_2 is $-\pi/2$ (axis into page)



$$\boldsymbol{x_{ee} = -l_1 sin(q_1) - l_2 sin(q_1 + q_2)}$$
$$\boldsymbol{y_{ee} = -l_1 cos(q_1) - l_2 cos(q_1 + q_2)}$$

Your turn: derive the Jacobian!

Implement the Impedance Controller

- Code at https://gitlab.epfl.ch/bellegar/lr-leg
- Upload your working code in Assignment 02 (1 per team)
- Today's assignment is not graded